visCOS

Daniel Klotz, Johannes Wesemann, Simon Frey, Mathew Herrnegger

Contents

1 The Guide **1**
1.1 Introduction . . . 2
1.2 visCOS and other R-package . . . 2

2 cosdata **2**
2.1 Examples plots from cosdata . . . 4

3 Aggregate according to time **6**
3.1 Code: `aggregate_time` . . . 10

4 Distance Metrics **11**
4.1 Code . . . 11
4.2 References . . . 17

5 Objective Function Summaries **17**
5.1 Code . . . 20

1 The Guide

This is the guide and definition of visCOS.

visCOS stands for "visual comparison of observations and simulations". The package is still changing quite often. We therefore recommend to not use it yet.

1.1 Introduction

This guide defines the code for the R-package visCOS and provides instructions, examples and tips for using it.

The goal of visCOS is to provide a possibility to streamline the R based visual-analysis for the hydrological model COSERO. COSERO is a HBV-like distributed rainfall-runoff model, developed at the institute for water management, hydrology and hydraulic engineering at BOKU, Vienna. The name is an abbreviation for "Conceptual Semi-Distributed Rainfall Runoff Model".

visCOS is embedded into a broader effort that aims to establish practical and easy-to-use workflows between R and COSERO. The overall concept of that project is shown in **Figure xyz**. The R-package `coscos` provides the backbone of the projects. It provides the basic interface between COSERO and R, generic helper functions and primitives, and defines the options for all subsequent package. Currently, visCOS is the only package that builds on it, but hopefully
soon we will have more packages, ideas and scripts that are able to use it. s

Figure xyz

1.2 visCOS and other R-package

There exist many R-packages dedicated to hydrology.

Among then, several have provide help to the calibration of (conceptual) rainfall-runoff models. As their name suggests, these models relate the a rainfall input to a generated runoff for some given basins They have found a wide array of applications in hydrology. Form the analysis of catchment properties to climate impact studies, so to speak. From the point o view of the R package ecosystem there are to packages, which are of particular interest with regard to visCOS: The `hydroGOF` package and the `hydroTSM` package. The former provides the most commonly used (and discussed) objective functions. The latter, helps with the analysis, interpolation and the plotting of hydrological time series.

As an R-package that aim of visCOSis to provide visual aids for the parameter estimation process of multiple (related) basins or catchments.

Thus, topic wise, visCOS can be positioned somewhere in-between the two previously mentioned packages. visCOS is usable for conceptual rainfall runoff models in general, but almost all of its functionality has been derived from different applications of the COSERO model.

2 cosdata

For visCOS raw-data is data directly taken from COSERO. In general, raw-data can have many forms. The focus of visCOS are time series of observations, o, and model simulations, s. It assumes that the data is already loaded in to R.

R has many options to read data. Most file-formats can be accessed with the native `read.` functions (for an example enter `?read.table` in your R-terminal). However, for larger files we recommend the read functionality of the `data.table` package. In our tests it was fastest and most flexible choice. Lastly, we recommend `readr` package for well structured and large data.

After the data is read into R it needs to be transformed (*cooked*) into a rigidly defined format, which we call **cosdata**. The format has some redundant columns, but provides a large optionality. It is also a good middle ground between the output of COSERO and the formats used within the R ecosystem.

All cosdata formated data can be used for further exploratory analysis with visCOS.

NOW WE NEED TO EXPLAIN COSDATA!

Snipptes:

Currently visCOSonly allows to compare between *numbered catchments*! The data **must** include an integer number at the end of its name, e.g. `QObs_001` and `QSim_001`).

Additionally, `cos_data` data.frame needs to have two different definitions for the date of a given row (see: Introduction). One is based on an old way to write dates out in fortran code. In that case each column represents a time-resolution. In concrete, the following columns are used:

- *yyyy* - year,
- *mm* - month,
- *hh* - hour,
- *min* - minute.

The other format is a more modern way to define time information. That is, the `POSIXct` format (see: link). This format is a standard for `R` and has many usages (e.g. transforming your data frame into a time series).

visCOS can differentiate seasonal information on a monthly resolution. The in- and out-of-period markings are stored in a separate column (defined by `viscos_options("name_COSperiod")`). Here, each season is defined by an number (integer), which starts 1 and is raised for each new season. A 0 indicates the out-of-period rows.

If needed, the column-names of the cosdata can adapted by using the `viscos_options` function from `coscos` (alternatively one might prefer to set the column-name the `data.frame` accordingly). Calling the function `viscos_options()` will return a list with the available options. The available options are as following:

```
# showing the options:
library(coscos)
library(magrittr)
viscos_options() %>% unlist(.)
```

```
##         name_o         name_s        name_lb        name_ub   name_COSyear
##         "qobs"         "qsim"           "lb"           "ub"         "yyyy"
##  name_COSmonth    name_COSday   name_COShour    name_COSmin  name_COSposix
##           "mm"           "dd"           "hh"          "min"    "posixdate"
## name_COSperiod      data_unit   missing_data        color_o        color_s
##       "period"      "(m^3/s)"         "-999"   "dodgerblue"       "orange"
##     of_limits1     of_limits2
##            "0"            "1"
```

An option can be changed by providing the new option-value as functional argument of the option. Here is an example for changing the *o* column color:

```
viscos_options(color_o = "dodgerblue")
viscos_options( ) %>% unlist(.)
```

```
##         name_o         name_s        name_lb        name_ub   name_COSyear
##         "qobs"         "qsim"           "lb"           "ub"         "yyyy"
##  name_COSmonth    name_COSday   name_COShour    name_COSmin  name_COSposix
##           "mm"           "dd"           "hh"          "min"    "posixdate"
## name_COSperiod      data_unit   missing_data        color_o        color_s
##       "period"      "(m^3/s)"         "-999"   "dodgerblue"       "orange"
##     of_limits1     of_limits2
##            "0"            "1"
```

The `coscos` package also provides a function to get some exemplary data to use in visCOS. Throughout this guide we will use it to demonstrate the different functionalities of visCOS. The data can be loaded with the function `viscos_example`, e.g.:

```
options(width=80)
require(coscos)
#
```

```r
raw_data_example <- viscos_example( )
head(raw_data_example)
```

```
##   yyyy mm dd hh min QOBS_0001 QOSI_0001 QSIM_0001 QOBS_0002 QOSI_0002 QSIM_0002
## 1 2007  1  1  0   0      2.98      3.48      3.48      2.56      3.11      3.11
## 2 2007  1  1  1   0      2.89      3.48      3.48      2.56      3.11      3.11
## 3 2007  1  1  2   0      2.64      3.48      3.48      2.57      3.11      3.11
## 4 2007  1  1  3   0      2.51      3.48      3.48      2.57      3.11      3.11
## 5 2007  1  1  4   0      2.42      3.48      3.48      2.57      3.11      3.11
## 6 2007  1  1  5   0      2.34      3.49      3.49      2.57      3.11      3.11
```

One can see that the example data still has some columns that are not needed for further analysis, i.e. the columns `QOSI_0001` and `QOSI_0002`. Furthermore, the `posixdate` and the `period` columns are still missing in the data. Again, the `coscos` provides a streamlined function, called `cook_cosdata` to get rid of the non-needed columns, complete the date columns and add the `period` column with pre-specified settings. With the raw-data example from before, the finished cosdata would look as following:

```r
raw_data_example %>%
  cook_cosdata(.) %>%
  head(.)
```

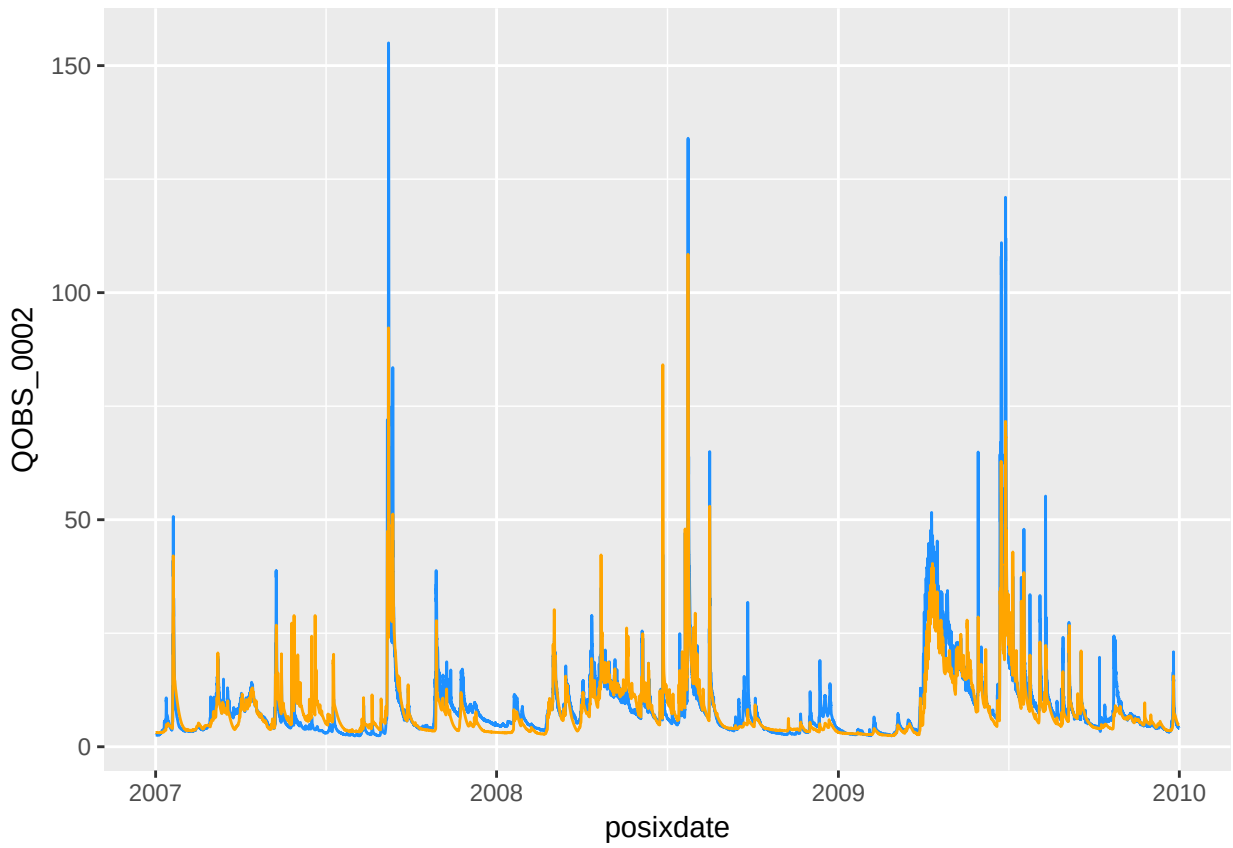
```
## # A tibble: 6 x 11
##    yyyy    mm    dd    hh   min QOBS_0001 QSIM_0001 QOBS_0002 QSIM_0002
##   <int> <int> <int> <int> <int>     <dbl>     <dbl>     <dbl>     <dbl>
## 1  2007     1     1     0     0      2.98      3.48      2.56      3.11
## 2  2007     1     1     1     0      2.89      3.48      2.56      3.11
## 3  2007     1     1     2     0      2.64      3.48      2.57      3.11
## 4  2007     1     1     3     0      2.51      3.48      2.57      3.11
## 5  2007     1     1     4     0      2.42      3.48      2.57      3.11
## 6  2007     1     1     5     0      2.34      3.49      2.57      3.11
## # ... with 2 more variables: posixdate <dttm>, period <int>
```

The `cook_cosdata` function can be very useful. If you are not sure that your data has the right format yet, just try to use `cook_cosdata` on it. The function is used at the start of almost all visCOS functions. Thus, it is very likely that your data is fine if `cook_cosdata` works on it.

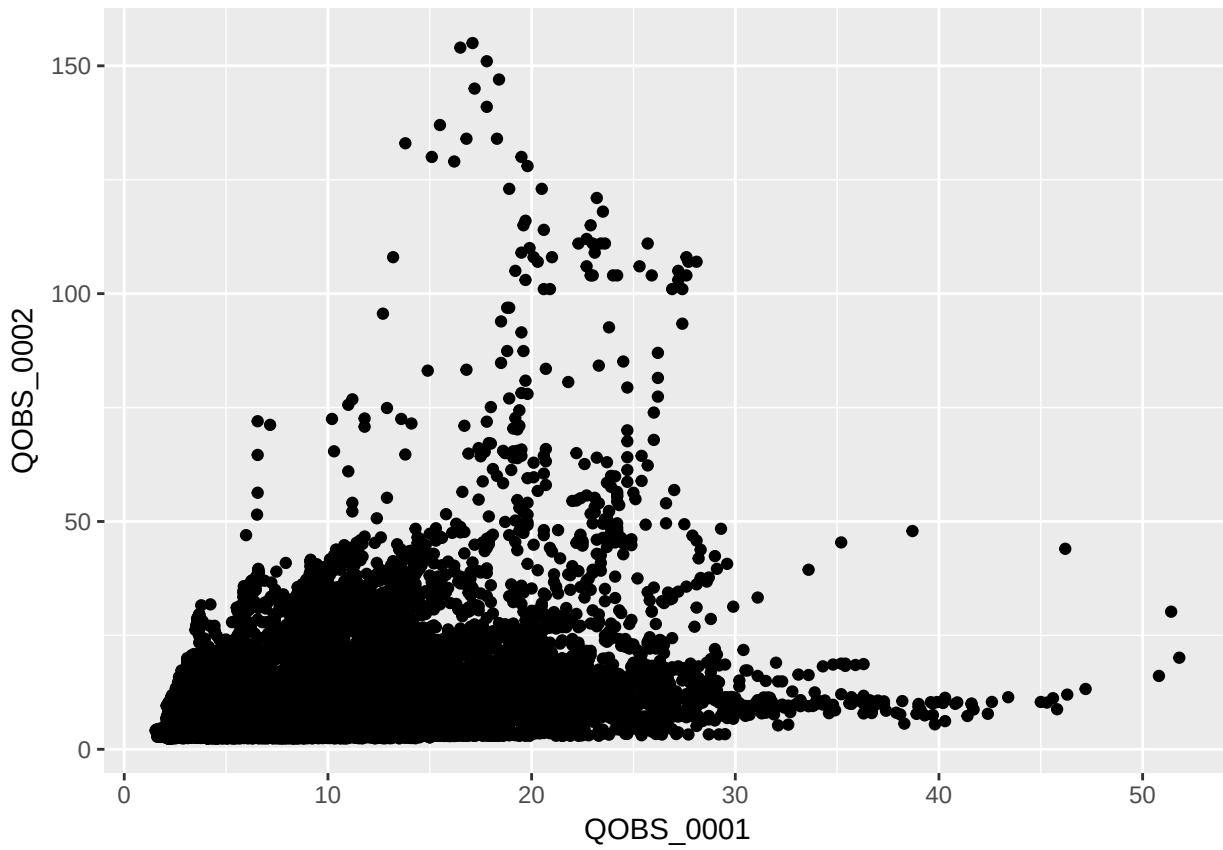
2.1 Examples plots from cosdata

It is already possible to do a lot with just the cosdata. In the following are some examples:

```r
library(ggplot2)
raw_data_example %>%
  cook_cosdata(.) %>%
  ggplot(.) +
    geom_line(aes(x = posixdate, y = QOBS_0002),
              color = viscos_options("color_o")) +
     geom_line(aes(x = posixdate, y = QSIM_0002),
              color = viscos_options("color_s"))
```




```
library(ggplot2)
raw_data_example %>%
  cook_cosdata(.) %>%
  ggplot(.) +
    geom_point(aes(x = QOBS_0001, y = QOBS_0002))
```



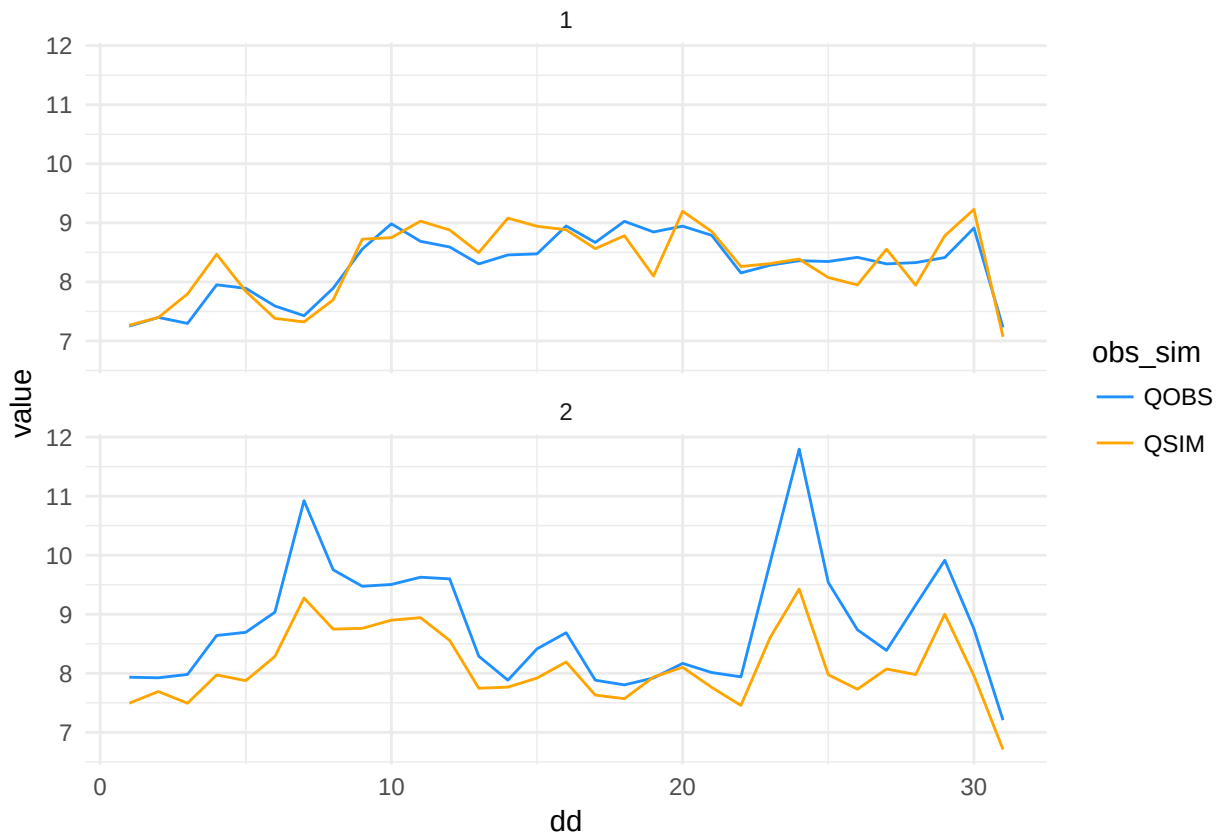
3 Aggregate according to time

In hydrology it is often useful to summarize the data respect to a given time dimension. In visCOS this can be done by using the `aggregate_time` function. The function takes in cosdata and aggregates them according to a chosen time dimension. In the following are some examples of its use:

```
coscos::libraries(visCOS, ggplot2, dplyr, magrittr)
```

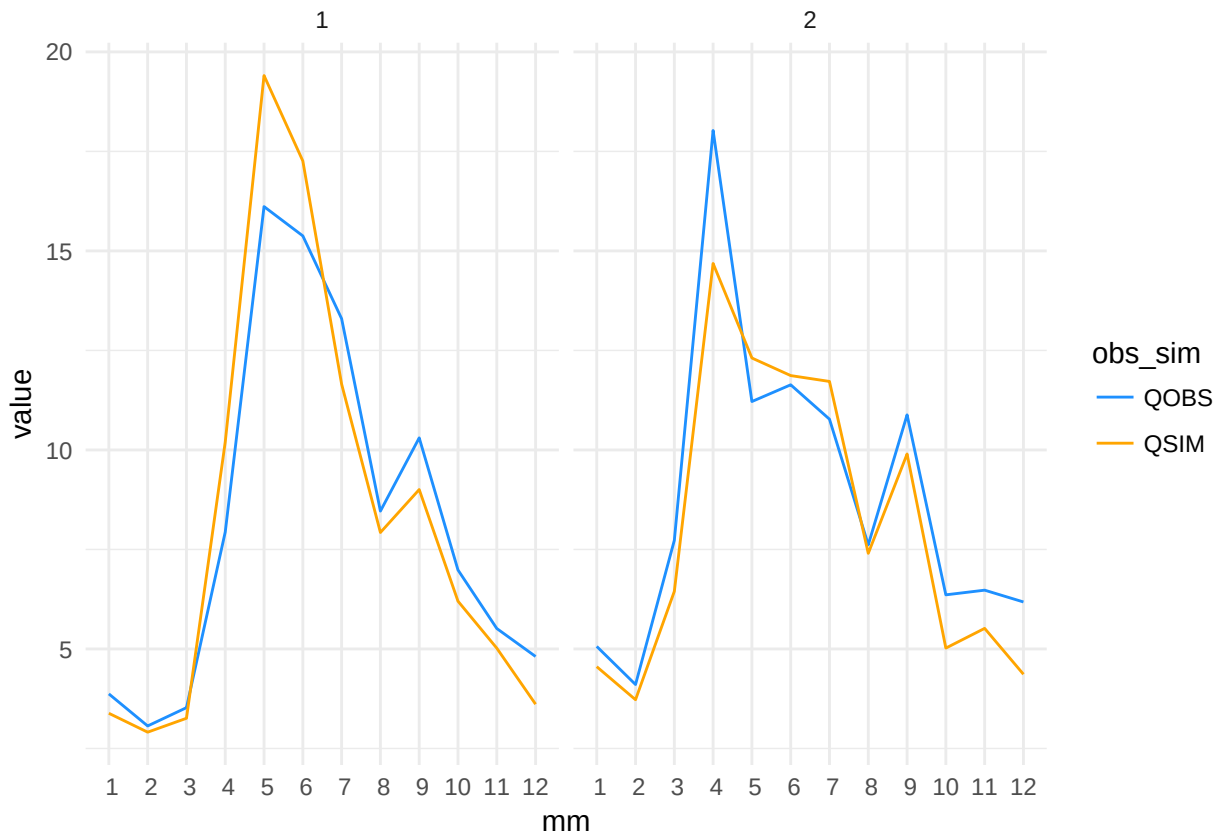
Daily runoff aggregation:

```
cosdata <- coscos::viscos_example()
runoff_aggregate_dd <- aggregate_time(cosdata, "dd") %>%
  dplyr::mutate(obs_sim = substr(key,1,4),
                basin = substr(key,6,9) %>% as.integer(.))
# plot data:
ggplot(runoff_aggregate_dd) +
    geom_line(aes(x = dd, y = value, col = obs_sim)) +
    scale_colour_manual(values = c(viscos_options("color_o"),
                                   viscos_options("color_s"))) +
    facet_wrap( ~ basin,ncol = 1) +
    theme_minimal()
```



Monthly runoff aggregation:

```
runoff_aggregate_mm <- aggregate_time(cosdata, "mm") %>%
  dplyr::mutate(obs_sim = substr(key,1,4),
                basin = substr(key,6,9) %>% as.integer(.))
# plot data:
ggplot(runoff_aggregate_mm) +
    geom_line(aes(x = mm, y = value, col = obs_sim)) +
    scale_colour_manual(values = c(viscos_options("color_o"),
                                   viscos_options("color_s"))) +
    scale_x_discrete(limits = runoff_aggregate_mm$mm) +
    facet_wrap( ~ basin) +
    theme_minimal()
```

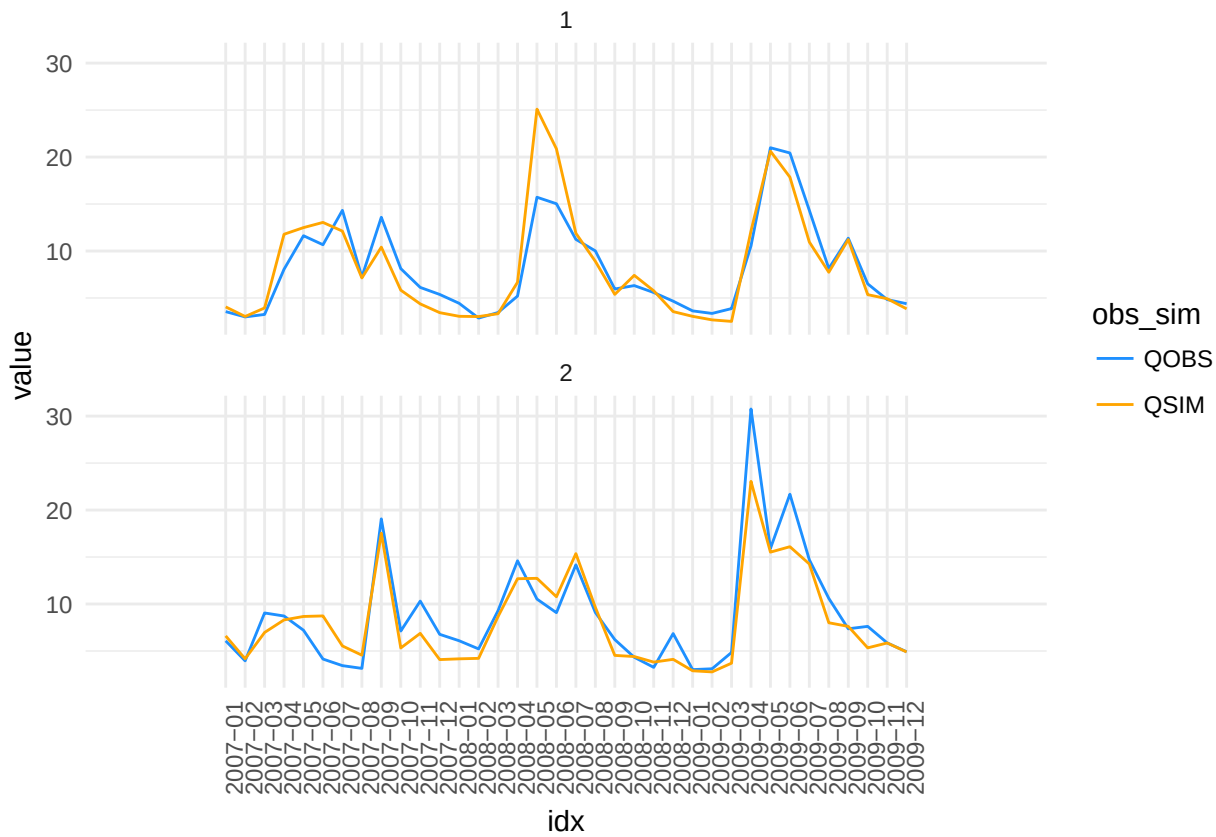


Yearly runoff aggregation:

```
runoff_aggregate_yyyy <- aggregate_time(cosdata, "yyyy") %>%
  dplyr::mutate(obs_sim = substr(key,1,4),
                basin = substr(key,6,9) %>% as.integer(.))
# plot data:
ggplot(runoff_aggregate_yyyy) +
  geom_line(aes(x = yyyy, y = value, col = obs_sim)) +
  geom_point(aes(x = yyyy, y = value, col = obs_sim)) +
  scale_colour_manual(values = c(viscos_options("color_o"),
                                 viscos_options("color_s"))) +
  facet_wrap( ~ basin) +
  scale_x_discrete(limits = runoff_aggregate_yyyy$yyyy,
                   labels = abbreviate) +
  theme_minimal()
```

Yearly and monthly runoff aggregation:

```
runoff_aggregate_yyyymm <- aggregate_time(cosdata, c("yyyy","mm")) %>%
  dplyr::mutate(obs_sim = substr(key,1,4),
                basin = substr(key,6,9) %>% as.integer(.),
                idx = yyyy + mm/12) # a small hack to get a unique identifier
# plot data:
ggplot(runoff_aggregate_yyyymm) +
  geom_line(aes(x = idx, y = value, col = obs_sim)) +
  scale_colour_manual(values = c(viscos_options("color_o"),
                                 viscos_options("color_s"))) +
  scale_x_discrete(limits = runoff_aggregate_yyyymm$idx,
                   labels = runoff_aggregate_yyyymm$posixdate) +
  facet_wrap( ~ basin,ncol = 1) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 90, hjust = 1))
```



3.1 Code: `aggregate_time`

The function `aggregate_time` is a wrapper around the `coscos` function `clump`. Here is the code:

```
#' Time Aggregation
#'
#' Aggregates the COSERO data.frame (\code{cosdata}) according to the
#' timely resolution defined via \code{aggregation}. Possible
#' resolution-choices are \code{'yyyy'} - year, \code{'mm'} - month and
#' \code{'dd'} - day and combinations thereof.
#'
#' @param cosdata The strictly defined data format (\code{cosdata}) used
#'          within \pkg{viscos}
#' @param aggregation A string that defines the resolution of the aggregation.
#'
#' @import coscos
#' @import pasta
#' @importFrom tidyr gather_
#' @export
aggregate_time <- function(cosdata,
                           key = "mm",
                           .funs = base::mean,
                           opts = coscos::viscos_options()) {
  if (class(key) != "character")
    stop("key must be a chracter. It currently is:" %&&% class(key))
  le_data <- coscos::cook_cosdata(cosdata)
  le_aggr <- coscos::clump(le_data, key = key, .funs = .funs)
```

```
    le_aggr[opts$name_COSposix] <- le_data[[opts$name_COSposix]] %>%
      coscos::clump_posix(.,key = key)
    le_names <- names(le_aggr)
    # melt the data in a tidy format:
    selected_cols <- (opts$name_o %|% opts$name_s) %>%
      grepl(.,le_names, ignore.case = TRUE) %>%
      le_names[.]
    tidy_aggr <- tidyr::gather_(le_aggr,
                                key_col = c("key"),
                                value_col = "value",
                                gather_cols = selected_cols)
    return(tidy_aggr)
  }
```

4 Distance Metrics

Metric play an important role in hydrology. Objective functions, $f_{of}(\cdot)$, are an important part of the hydrological model calibration. This chapter defines the objective functions that are provided by visCOS.

The importance of metrics in hydrology seems to arise ultimately from the approximate nature of the subject and the the large uncertainties involved in the task that hydrologist try to solve. Over the years a many different much ink has been spilled for discussing the different merits and pitfalls of individual objective functions. Thus, with time many different ones have been proposed. Some are more useful for model comparison and some objective functions try to address specific problems of other ones.

4.1 Code

This section defines the code for different objective functions. FORTRAN Routines are used for most of them. The results are slightly different then the ones from the R-code of the `hydroGOF` package (~13th decimal place), but always faster. For the explanation and definition of the objective function it is assumed that o are the observations (defined by `name_o` in visCOS) and s are the simulation(defined by `name_s` in visCOS).

```
#' Distance Measures (Metrics)
#'
#' Different objective Functions, provided by visCOS. A detailed description
#' of each of the provided objective function is provided in the respective
#' vignette
#'
#' @useDynLib visCOS
#' @param o The reference data or observations (o_data)
#' @param s The created data or the simulations (s_data)
#' @name d_metrics
NULL
```

4.1.1 The "Main" Objective Functions

Currently the main objective functions are the Nash-Sutcliffe Efficiency, the Kling-Gupta Efficiency, the percentage bias and the correlation.

4.1.1.1 Nash-Sutcliffe Efficiency

The Nash-Sutcliffe Criterion NSE is by far the most used efficiency criterion in hydrology. In the hydrological context o usually represents a set of runoff-observation and s a set of simulations. The NSE is defined in the same way as the general definition of the coefficient of determination R^2:

$$NSE = \frac{\sum_{t=1}^{T} \big(o(t) - s(t)\big)^2}{\sum_{t=1}^{T} \big(o(t) - \bar{o}\big)^2}.$$

The variable $\bar{o}$ represents the average of o. The NSE can be seen as the relational the estimator s and the estimator resulting form the average of the data. It can can have values between minus infinity and 1; with 1 being the perfect fit, 0 when the mean of s is as good as the mean of o and negative values are even worse.

The code for the NSE computation is:

```
# ----------------------------------------------------------------------
#' Nash-Sutcliffe Efficiency
#'
#' @rdname d_metrics
#' @importFrom coscos build_tibble
#' @importFrom purrr map2_dbl
#' @importFrom magrittr set_names
#' @export
d_nse <- function(o, s, na.rm = TRUE) {
  # pre sets:
  o <- as_tibble(o)
  s <- as_tibble(s)
  rows <- nrow(s) %>% as.integer(.)
  cols <- ncol(s) %>% as.integer(.)
  if(rows != nrow(o)) stop("o and s must have the same amount of rows (data points)")
  if(cols != ncol(o)) stop("o and s must have the same amount of columns (variables)")
  # computation:
  of_nse <- map2_dbl(o,
                     s,
                     function(x,y) d_wrapper(x, y, rows, d_name = "f_nse", na.rm = na.rm)) %>%
    set_names(paste("nse", 1:cols, sep = ""))
  return(of_nse)
}
```

4.1.1.2 Kling-Gupta Efficiency

The Kling-Gupta Efficiency, KGE, was introduced by Gupta et al. (2009) to alleviate some of the shortcomings of the NSE. In their paper they argue why the NSE tends to overate simulations with small variance (note: in the context of the paper simulations $= s$) and propose their efficiency criterion instead.

The KGE is defined as:

$$KGE = 1 - ED,$$

with

$$ED = \sqrt{\big(corr(o,s) - 1\big)^2 + \big(\alpha(o,s) - 1\big)^2 + \big(\beta(o,s) - 1\big)^2}.$$

In which $\alpha(o,s) = \frac{\sigma_s}{\sigma_o}$ is the standard deviation σ of s divided by the σ of o, $\beta(o,s) = \mu_s/\mu_o$ withμ being the arithmetic mean and $corr(o,s)$ as the Pearson's correlation coefficient (see below). The value range and the quality is similar to the NSE.

The code for the KGE computation is:

```
# ----------------------------------------------------------------------------
#' KGE 2
#'
#' @rdname d_metrics
#' @importFrom coscos build_tibble
#' @export
d_kge <- function(o, s, na.rm = TRUE) {
  # pre sets:
  o <- as_tibble(o)
  s <- as_tibble(s)
  rows <- nrow(s) %>% as.integer(.)
  cols <- ncol(s) %>% as.integer(.)
  if(rows != nrow(o)) stop("o and s must have the same amount of rows (data points)")
  if(cols != ncol(o)) stop("o and s must have the same amount of columns (variables)")
  # computation:
  of_kge <- map2_dbl(o,
                     s,
                     function(x,y) d_wrapper(x, y, rows, na.rm = na.rm)
    ) %>%
    set_names(paste("kge", 1:cols, sep = ""))
  return(of_kge)
}
```

4.1.1.3 Bias

The bias p_{bias} is defined as the sum of the differences between s and o divided by the sum of o:

$$p_{bias} = \frac{\sum_{t=1}^{T}[o(t) - s(t)]}{T}.$$

The code for the p_{bias} computation is:

```
# ----------------------------------------------------------------------------
#' Percentage Bias
#'
#' @rdname d_metrics
#'
#' @importFrom coscos build_tibble
#'
#' @export
d_bias <- function(o, s, na.rm = TRUE) {
  # pre sets:
  o <- as_tibble(o)
  s <- as_tibble(s)
  rows <- nrow(s) %>% as.integer(.)
  cols <- ncol(s) %>% as.integer(.)
  if(rows != nrow(o)) stop("o and s must have the same amount of rows (data points)")
  if(cols != ncol(o)) stop("o and s must have the same amount of columns (variables)")
  # computation:
```

```
  of_pbias <- map2_dbl(o, s,
                       function(x,y) d_wrapper(x, y, rows, d_name = "f_bias", na.rm = na.rm)) %>%
    set_names(paste("bias", 1:cols, sep = ""))
  return(of_pbias)
}
```

4.1.1.4 Percentage Bias

The percentage of bias p_{bias} is defined as the sum of the differences between s and o divided by the sum of o and multiplied by 100:

$$p_{bias} = 100 * \frac{\sum_{t=1}^{T}[s(t) - o(t)]}{\sum_{t=1}^{T} o(t)}.$$

The code for the p_{bias} computation is:

```
# ----------------------------------------------------------------------
#' Percentage Bias
#'
#' @rdname d_metrics
#'
#' @importFrom coscos build_tibble
#'
#' @export
d_pbias <- function(o, s, na.rm = TRUE) {
  # pre sets:
  o <- as_tibble(o)
  s <- as_tibble(s)
  rows <- nrow(s) %>% as.integer(.)
  cols <- ncol(s) %>% as.integer(.)
  if(rows != nrow(o)) stop("o and s must have the same amount of rows (data points)")
  if(cols != ncol(o)) stop("o and s must have the same amount of columns (variables)")
  # computation:
  of_pbias <- map2_dbl(o, s,
                       function(x,y) d_wrapper(x, y, rows, d_name = "f_pbias", na.rm = na.rm)) %>%
    set_names(paste("pbias", 1:cols, sep = ""))
  return(of_pbias)
}
```

4.1.1.5 Pearson's correlation coefficient

Pearson's correlation coefficient, r or $corr(o, s)$, is a measure of the linear relationship between o and s. It is defined as:

$$r = \frac{cov(o, s)}{\sigma_s * \sigma_o},$$

where $cov(...)$ denotes the covariance. The correlation coefficient can take on values between -1 and 1. The former corresponds to an inverse and the latter to a direct relationship and the closer the values is to zero the weaker is the implied correlation.

The code for the correlation is:

```r
# ----------------------------------------------------------------------------
#' Correlation
#'
#' @rdname d_metrics
#'
#' @importFrom coscos build_tibble
#'
#' @export
d_cor <- function(o, s) {
  # pre sets:
  o <- as_tibble(o)
  s <- as_tibble(s)
  rows <- nrow(s) %>% as.integer(.)
  cols <- ncol(s) %>% as.integer(.)
  if(rows != nrow(o)) stop("o and s must have the same amount of rows (data points)")
  if(cols != ncol(o)) stop("o and s must have the same amount of columns (variables)")
  # computation:
  stats::cor(o, s) %>% diag(.)
}
```

4.1.2 Mean Squared Error

Defined as:

$$MSE = \frac{\sum_{t=1}^{T} \big(o(t) - s(t)\big)^2}{T}$$

```r
# ----------------------------------------------------------------------------
#' Mean Squared Error
#'
#' @rdname d_metrics
#'
#' @importFrom coscos build_tibble
#'
#' @export
d_mse <- function(o, s, na.rm = TRUE) {
  # pre sets:
  o <- as_tibble(o)
  s <- as_tibble(s)
  rows <- nrow(s) %>% as.integer(.)
  cols <- ncol(s) %>% as.integer(.)
  if(rows != nrow(o)) stop("o and s must have the same amount of rows (data points)")
  if(cols != ncol(o)) stop("o and s must have the same amount of columns (variables)")
  # computation:
  of_mse <- map2_dbl(o, s,
                     function(x,y) d_wrapper(x, y, rows, d_name = "f_mse", na.rm = na.rm)) %>%
    set_names(paste("mse", 1:cols, sep = ""))
  return(of_mse)
}
```

4.1.2.1 Root Mean Squared Error

The square root of the MSE

```r
# ------------------------------------------------------------------------
#' Root Mean Squared Error
#'
#' @rdname d_metrics
#'
#' @importFrom coscos build_tibble
#'
#' @export
d_rmse <- function(o, s, na.rm = TRUE) {
  cols <- ncol(s) %>% as.integer(.)
  d_mse(o,s,na.rm) %>%
    sqrt(.) %>%
    set_names(paste("rmse", 1:cols, sep = "")) %>%
    return(.)
}
```

4.1.2.2 Inverted Nash-Sutcliffe Efficiency

$$inse = \frac{\sum_{t=1}^{T} \left(s(t) - o(t)\right)^2}{\sum_{t=1}^{T} \left(s(t) - \bar{s}\right)^2}$$

```r
# ------------------------------------------------------------------------
#' Inverted Nash-Sutcliffe Efficiency
#'
#' @rdname d_metrics
#'
#' @importFrom coscos build_tibble
#'
#' @export
d_inse <- function(o, s, na.rm = TRUE) {
  cols <- ncol(s) %>% as.integer(.)
  d_nse(o, s, na.rm = na.rm) %>%
    set_names(paste("inse", 1:cols, sep = "")) %>%
    return(.)
}
```

4.1.3 Distance Function Fortran Wrapper

```r
d_wrapper <- function(obs, sim, rows, ndstart = 1L, ndend = rows, d_name = "f_kge", na.rm) {
  idx_to_eval <- as.integer( 1L - (is.na(obs) + is.na(sim)) )
  na_count <- sum(idx_to_eval)
  if (!na.rm & (na_count > 0)) {
    stop("There are NAs in the data but `na.rm` is set to `FALSE`!")
  }
  out <- .Fortran(d_name,
                  XSIM = as.double(sim),
                  XOBS = as.double(obs),
                  maxday = rows,
                  NDSTART = 1L,
                  NDEND = rows,
                  EVAL = idx_to_eval,
```

```
                    of = as.double(-999.9),
                    NAOK = TRUE)
      return(out$of)
  }
```

“‘

4.2 References

- **Percentage Bias:** Yapo P. O., Gupta H. V., Sorooshian S., 1996. Automatic calibration of conceptual rainfall-runoff models: sensitivity to calibration data. Journal of Hydrology. v181 i1-4. 23-48
- **Nash-Sutcliffe Efficiency:** Nash, J. E. and J. V. Sutcliffe (1970), River flow forecasting through conceptual models part I -A discussion of principles, Journal of Hydrology, 10 (3), 282-290
- **Kling-Gupta Efficiency:** Gupta, Hoshin V., Harald Kling, Koray K. Yilmaz, Guillermo F. Martinez. Decomposition of the mean squared error and NSE performance criteria: Implications for improving hydrological modelling. Journal of Hydrology, Volume 377, Issues 1-2, 20 October 2009, Pages 80-91. DOI: 10.1016/j.jhydrol.2009.08.003. ISSN 0022-1694
- **Volumetric Efficiency:** Criss, R. E. and Winston, W. E. (2008), Do Nash values have value? Discussion and alternate proposals. Hydrological Processes, 22: 2723-2725. doi: 10.1002/hyp.7072

5 Objective Function Summaries

This chapter presents the objective function summary functions in visCOS. They compute basin-wise objective functions of the overall data and for the individual basins.

As explained in previous section objective functions are a pivotal part of model calibration. If no specific options are chosen visCOS uses 4 objective functions, i.e.: NSE, KGE, Pearson's Correlation and the Percentage Bias (the function definitions have been given in the chapter about distance metrics).

The objective functions for the overall data and the marked periods can be computed through the function `judge_compute`. Additionally, visCOS already provides two different options to create plots for the main objective functions: `judge_rasterplot` and `judge_barplot`. Both functions create a list with `ggplot` figures. Each entry in the list corresponds to one of the main objective functions and both lists can be saved to html embedded pictures with the `serve` function.

The `judge` function provides a wrapper for all other `judge_` and `of_` functions. The selection is done via the non-standardly-evaluated `type` argument (search for lazy-evaluation if you want to know more. This implies, among other things, that

In the following are some examples:

Example 1. Computing the main objective functions

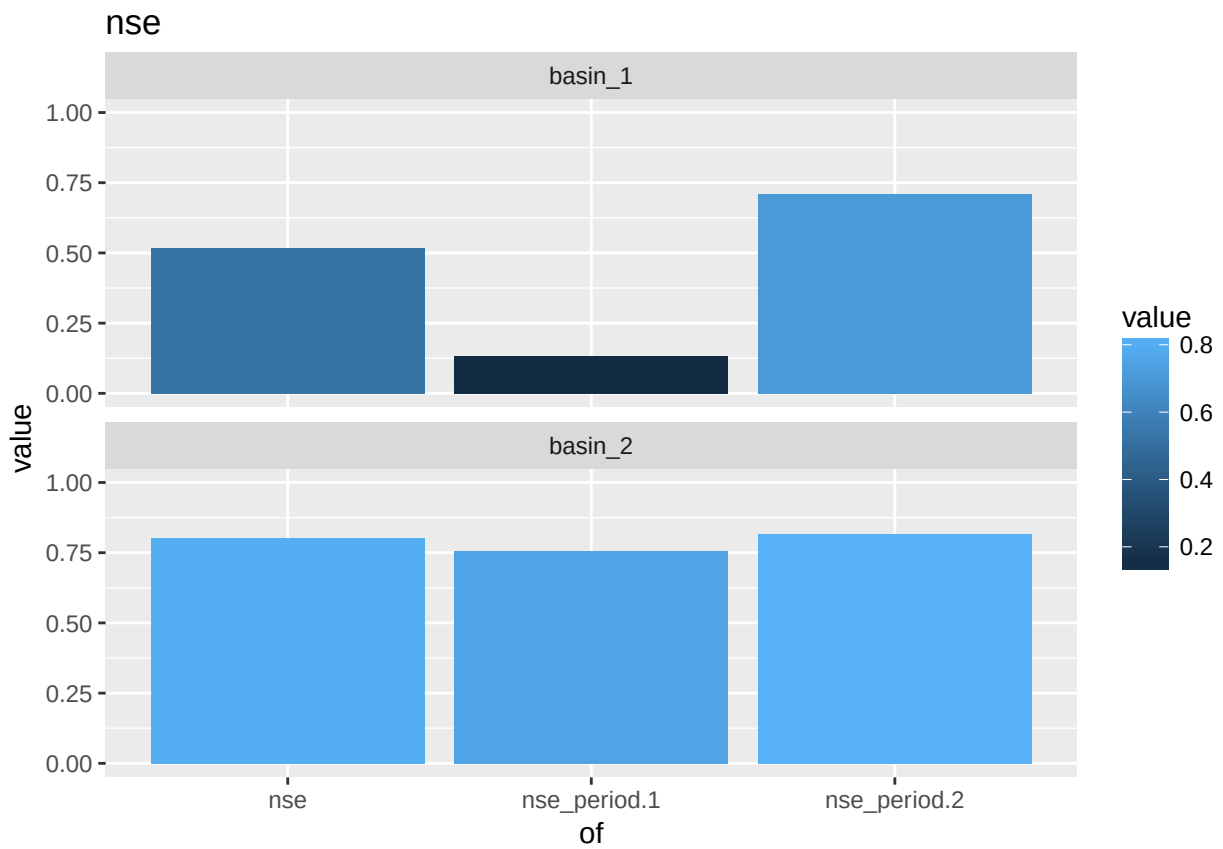
```
require(coscos)
require(visCOS)
require(magrittr)
#
viscos_example( ) %>%
  judge(.) %>%
  print(.)
```

```
## # A tibble: 12 x 3
##                of    basin_1     basin_2
##  *          <chr>      <dbl>       <dbl>
##  1            nse  0.5177309   0.8012919
```

```
## 2            kge  0.7246877   0.7489862
## 3          pbias  0.2506191 -12.5298276
## 4           corr  0.8177663   0.9107391
## 5  corr_period.1  0.8243198   0.8806770
## 6  corr_period.2  0.8577983   0.9385221
## 7   kge_period.1  0.4138710   0.8487406
## 8   kge_period.2  0.8419666   0.6820128
## 9   nse_period.1  0.1326604   0.7566200
## 10  nse_period.2  0.7096110   0.8168895
## 11 pbias_period.1  8.8238223  -8.6127526
## 12 pbias_period.2 -6.8569950 -15.9030884
```

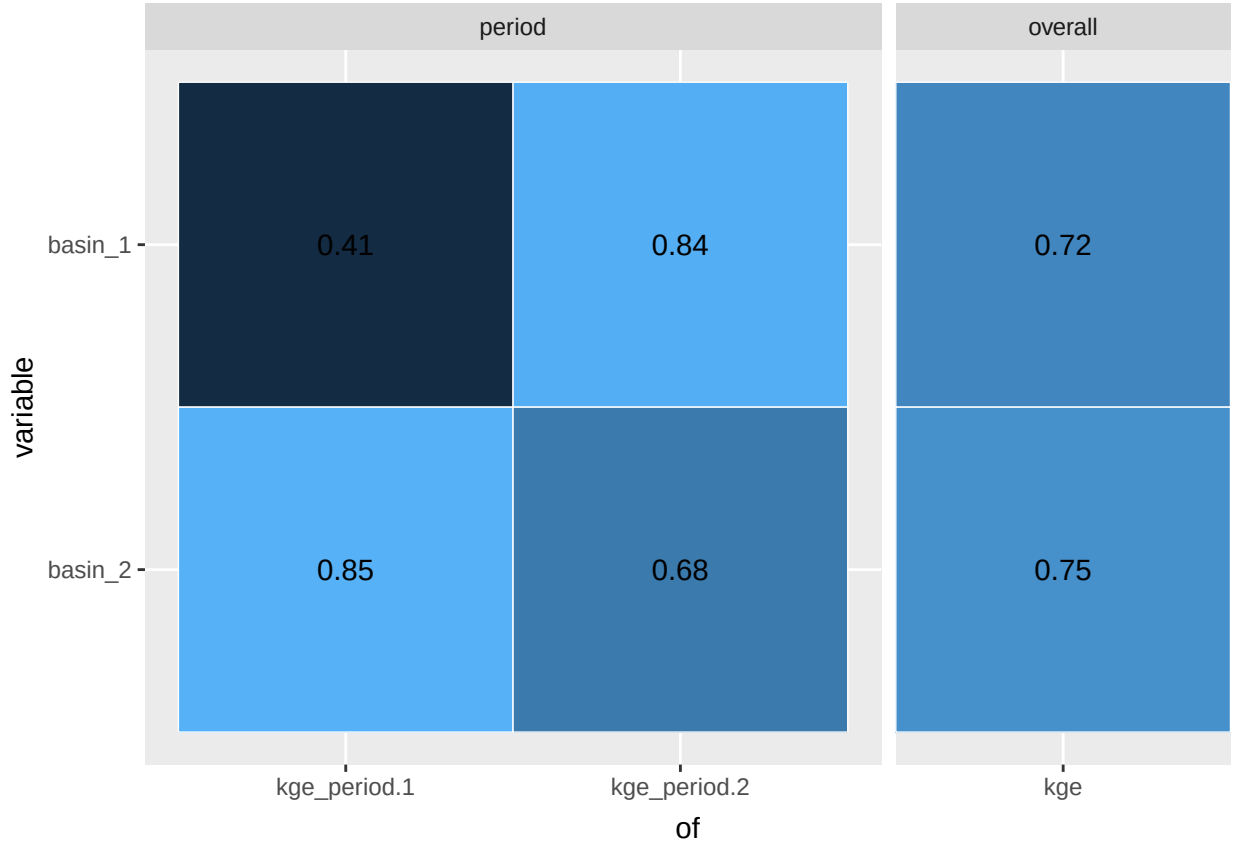
Example 2. Plotting the results of main objective function with bar plots:

```
viscos_example() %>%
  judge_barplot(.) %>%
  .[[1]] %>%
  plot(.)
```



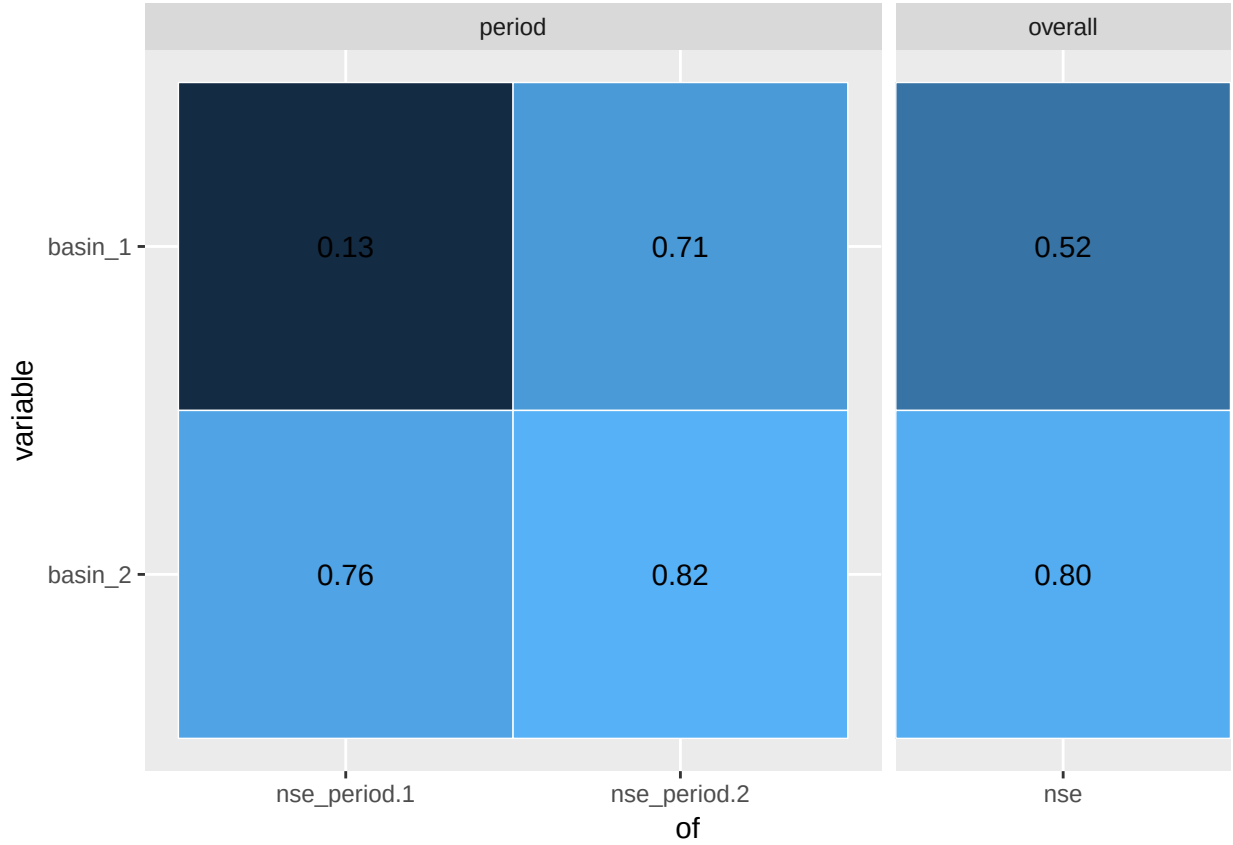
Example 3. Plotting the results of main objective function with a raster:

```
viscos_example() %>%
  judge_rasterplot(.) %>%
  .[[2]] %>%
  plot(.)
```



Example 4. Using the `judge` function as a wrapper:

```
viscos_example() %>%
  judge(.,rasterplot) %>%
  .[[1]] %>%
  plot(.)
```



5.1 Code

```
# ---------------------------------------------------------------------------
# Code for the Main Objective Functions (main_of)
# authors: Daniel Klotz, Johannes Wesemann, Mathew Herrnegger
# !!!!
```

```
                  d_metrics = list(nse = d_nse,
                                   kge = d_kge,
                                   pbias = d_pbias,
                                   corr = d_cor),
                  opts = coscos::viscos_options(),
                  ...) {
  # lazy evaluation: -------------------------------------------------------
  le_dots <-  match.call(expand.dots = FALSE) %>%
    .["..."] %>%
    .[[1]]
  le_type <- match.call(expand.dots = FALSE) %>%
    .["type"] %>%
    .[[1]] %>%
    as.character(.)
  if (length(le_type) == 0) {
    le_type = "compute"
  }
  if (class(le_type) != "character") {
    stop("Cannot resolve 'type' argument. Try to provide a character.")
  }
  # compute options: -------------------------------------------------------
  if (le_type == "compute") {
    judge_compute(cosdata, d_metrics)
  } else if (le_type == "judge_barplot" | le_type == "barplot") {
    judge_barplot(cosdata, d_metrics)
  } else if (le_type == "judge_rasterplot" | le_type == "rasterplot") {
    judge_rasterplot(cosdata, d_metrics)
  } else if (le_type == "of_explore" | le_type == "explore") {
    of_explore(cosdata, d_metrics)
  } else if (le_type == "of_compare" | le_type == "compare") {
    cosdata2 <- NULL
    if ("cos_data2" %in% names(le_dots)) {
      cos_data2 <- eval(le_dots$cos_data2)
    }
    of_compare(d1 = cosdata,
               d2 = cosdata2)
  } else {
    stop("There is no option (`type`) called" %&&% le_type)
  }
}
```

5.1.2 Computation

The purpose of this function is to extract a the main of objective functions from the `COSERO` data.frame. The objective functions are extracted for each basin separately and computed for the entire length of the data, as well as for each period separately.

The computational part of the function works as follows. In step (I) the non-marked periods of `cos_data` (columns of `viscos_options("name_COSperiod")` which are smaller then 0) are excluded from further calculations. The thereby obtained data.frame is named `evaluation_data`.

```
# ---------------------------------------------------------------------------
#' Get basic objective function for cos_data
#'
```

```r
#' Calculate basic objective functions(NSE, KGE, percentage BIAS, Correlation)
#' for every basin and the chosen periods.
#'
#' @param cos_data cos_data data.frame.
#' @return list of basic objective function evaluated for the different
#' hydrological years and over the whole timespan.
#'
#' @import pasta
#' @import coscos
#' @import tibble
#' @importFrom purrr map_df
#' @importFrom dplyr select filter starts_with
#' @importFrom magrittr set_names
judge_compute <- function(cosdata,
                        d_metrics = list(nse = d_nse,
                                         kge = d_kge,
                                         pbias = d_pbias,
                                         corr = d_cor),
                        opts = coscos::viscos_options()) {
  # pre: ====================================================================
  cos_data <- coscos::cook_cosdata(cosdata)
  name_period <- opts[["name_COSperiod"]]
  name_o <- opts[["name_o"]]
  name_s <- opts[["name_s"]]
  #
  if( !(class(d_metrics) == "list") ) {
    d_metrics <- list(d_metrics)
  }
  if (is.null(names(d_metrics))){
    d_names <- "d" %&% 1:length(d_metrics)
  } else {
    d_names <- names(d_metrics)
  }
  #
  evaluation_data <- cos_data[cos_data[[name_period]] > 0, ]
  number_judge_basins <- evaluation_data %>%
    names(.) %>%
    unique(.) %>%
    grepl(name_o, ., ignore.case = TRUE) %>%
    sum(.)
  data_periods <- evaluation_data %>%
    .[[name_period]] %>%
    unique(.)
  number_judge_periods <- data_periods %>% length

  # compute main-of for entire data: ========================================
  d_mean <- purrr::map_df(d_metrics, function(judge_, x, y) as.numeric(judge_(x,y)),
                 x = dplyr::select(evaluation_data,dplyr::starts_with(name_o)) %>% unname(.),
                 y = dplyr::select(evaluation_data,dplyr::starts_with(name_s)) %>% unname(.) ) %>%
    t(.) %>%
    as_tibble(.) %>%
    magrittr::set_names(., "basin" %_% 1:number_judge_basins) %>%
    cbind.data.frame(of = d_names,.)
```

```r
  # compute periodwise main-of: ==============================================
  period_compute <- function(k) {
    o_pick <- dplyr::filter(evaluation_data,period == data_periods[k]) %>%
      dplyr::select(.,starts_with(name_o)) %>%
      unname(.)
    s_pick <- dplyr::filter(evaluation_data,period == data_periods[k]) %>%
      dplyr::select(.,starts_with(name_s)) %>%
      unname(.)
    d_measures <- purrr::map_df(d_metrics, function(judge_,x,y) as.numeric(judge_(x,y)),
                 x = o_pick,
                 y = s_pick ) %>% t()
    return(measure = d_measures)
  }
  d_names_periods <- d_names %_% "period" %.% rep(1:number_judge_periods, each = length(d_names))
  d_periods <- lapply(1:number_judge_periods, period_compute) %>%
    do.call(rbind,.) %>%
    as_tibble(.) %>%
    magrittr::set_names("basin" %_% 1:number_judge_basins) %>%
    cbind(of = d_names_periods,.) %>%
    .[order(.$of), ]
  #
  judge_all <- rbind(d_mean,d_periods) %>%
    as_tibble(.)
  judge_all$of <- as.character(judge_all$of)
  return(judge_all)
}
```

5.1.3 Plotting

```r
# ------------------------------------------------------------------------------
#' Plot main objective function values
#'
#' Currently two options for plotting the main objectives are provided by
#' visCOS: Plotting the different objective functions values as a set of
#' bar plots \code{barplot_of} and plotting a summary table in form of
#' a large raster of all the objective function values \code{rasterplot_of}.
#'
#' @name plot_main_of
NULL
```

5.1.3.1 Bar Plots

```r
# ------------------------------------------------------------------------------
#' Bar plot for the Main Objective Function Values
#'
#' @rdname judge_overview
#'
#' @import coscos
#'
#' @export
judge_barplot <- function(cosdata, d_metrics = list(nse = d_nse,
                                                    kge = d_kge,
```

```r
                                        pbias = d_pbias,
                                        corr = d_cor)) {
  # pre: ==================================================================
  cos_data <- coscos::cook_cosdata(cosdata)
  if(!(class(d_metrics) == "list")) {
    d_metrics <- list(d_metrics)
  }
  if (is.null(names(d_metrics))){
    d_names <- "d" %&% 1:length(d_metrics)
  } else {
    d_names <- names(d_metrics)
  }
  # functions: ============================================================
  assign_ofgroups <- function(judge_melted,judge_names) {
    judge_string <- as.character(judge_melted$of)
    judge_melted$judge_group <- judge_string %>%
      replace(., startsWith(judge_string,judge_names[1]), judge_names[1]) %>%
      replace(., startsWith(judge_string,judge_names[2]), judge_names[2]) %>%
      replace(., startsWith(judge_string,judge_names[3]), judge_names[3]) %>%
      replace(., startsWith(judge_string,judge_names[4]), judge_names[4])
    return(judge_melted)
  }
  # plot-list function:
  barplot_fun <- function(judge_name,judge_melted) {
    judge_to_plot <- judge_melted %>% filter( judge_group == judge_name)
    # bad solution, but will be fine for now?
    if (judge_name == "pbias") {
      gglimits <- c(-viscos_options("of_limits")[2]*100,
                    viscos_options("of_limits")[2]*100)
    } else {
      gglimits <- viscos_options("of_limits")
    }
    plt_out <- ggplot(data = judge_to_plot) +
      geom_bar(stat = "identity",
               position = "identity",
               aes(x = of, y = value, fill = value)) +
      facet_wrap(~ variable, ncol = 1) +
      ggtitle(judge_name) +
      ylim(gglimits)
    return(plt_out)
  }
  # computations: =========================================================
  judge_data <- judge_compute(cos_data,d_metrics)
  num_basins <- ncol(judge_data) - 1
  judge_melted <- suppressMessages( reshape2::melt(judge_data) ) %>%
    assign_ofgroups(.,d_names)
  # plotting ==============================================================
  plot_list <- lapply(d_names, function(x) barplot_fun(x,judge_melted)) %>%
    magrittr::set_names(d_names)
  return(plot_list)
}
```

5.1.3.2 Raster Plots

```r
#' Bar plot for the Main Objective Function Values
#'
#' @rdname judge_overview
#' @import pasta
#' @export
judge_rasterplot <- function(cos_data, d_metrics = list(nse = d_nse,
                                        kge = d_kge,
                                        pbias = d_pbias,
                                        corr = d_cor)) {
  # def: ==================================================================
  build_tibble(cos_data)
  if(!(class(d_metrics) == "list")) {
    d_metrics <- list(d_metrics)
  }
  if (is.null(names(d_metrics))){
    d_names <- "d" %&% 1:length(d_metrics)
  } else {
    d_names <- names(d_metrics)
  }
  # computations: ========================================================
  regex_main_of <- d_names %.% "*"
  judge_data <- judge_compute(cos_data, d_metrics)
  #
  plot_list <- lapply(regex_main_of,function(x) plot_fun_raster(x,judge_data)) %>%
    magrittr::set_names(d_names)
  return(plot_list)
}

# plot function ---------------------------------------------------------
plot_fun_raster <- function(regex_single_of,judge_data) {
  # function definitions ================================================
  extract_single_of <- function(judge_data){
    idx <- grep(regex_single_of,judge_data$of)
    return(judge_data[idx, ])
  }
  add_facet_info <- function(judge_data) {
    facet_column <- nrow(judge_data) %>%
      magrittr::subtract(1) %>%
      rep("period",.) %>%
      c("overall",.)
    return( cbind(judge_data,facets = facet_column) )
  }
  reverse_basin_levels <- function(prepared_data) {
    prepared_data$variable <- factor(prepared_data$variable,
                                     levels = prepared_data$variable %>%
                                       levels() %>%
                                       rev()
                                     )
    return(prepared_data)
  }
  reverse_facetting_levels <- function(prepared_data) {
    prepared_data$facets <- factor(prepared_data$facets,
                                   levels = prepared_data$facets %>%
```

```
                                levels() %>%
                                rev()
                              )
    return(prepared_data)
  }
  bind_and_round_value <- function(of,gglimits,digits) {
    dplyr::mutate(of,
                  value = pmax(value,gglimits[1]) %>%
                    pmin(.,gglimits[2]) %>%
                    round(.,digits)
                  )
  }
  # computation ==============================================================
  if (regex_single_of == "p_bias.*") {
    # pbias has different limits :(
    gglimits <- c(-viscos_options("of_limits")[2]*100,
                  viscos_options("of_limits")[2]*100)
  } else {
    gglimits <- viscos_options("of_limits")
  }
  #
  prepared_data <- judge_data %>%
    extract_single_of() %>%
    add_facet_info() %>%
    reshape2::melt(., id.vars = c("of","facets")) %>%
    reverse_basin_levels() %>%
    reverse_facetting_levels() %>%
    bind_and_round_value(.,gglimits,2)
  # ggplot ===================================================================
  plt_out <- ggplot(prepared_data,
                    aes(of,variable, fill = value),
                    environmnet = environment()) +
    geom_raster(position = "identity") +
    coord_fixed(ratio = 5)  +
    facet_grid(~ facets,  scales = "free_x", space = "free") +
    theme( legend.position = "none")  +
    geom_tile(color = "white", size = 0.25 ) +
    geom_text(aes(of,variable, label = as.character(sprintf("%0.2f",value,2))),
              color = "black")
  return(plt_out)
}
```